

Aula 000 - Lista de Exercícios

Introdução aos Paradigmas de Programação

Orientações

Responda de maneira objetiva, mas justifique suas decisões. Nas questões com código, o foco não é decorar sintaxe: é identificar conceitos, comportamentos e formas de organizar a solução.

Parte A - Conceitos fundamentais

1. Por que estudar paradigmas?

Explique três benefícios práticos de estudar conceitos de linguagens de programação, mesmo quando o desenvolvedor trabalha diariamente com apenas uma linguagem.

Estudar paradigmas aumenta a capacidade de resolver problemas, pois amplia as formas de pensar e estruturar soluções. Também facilita o aprendizado de novas linguagens, já que os conceitos fundamentais são mais facilmente reconhecidos. Além disso, melhora o uso da linguagem já conhecida, permitindo aproveitar recursos que antes não eram utilizados.

2. Linguagem, paradigma e domínio

Defina com suas palavras: linguagem de programação, paradigma e domínio de aplicação. Depois, dê um exemplo que relacione os três conceitos

Uma linguagem de programação é uma forma de escrever instruções para um computador, um paradigma é um modelo ou estilo usado para organizar e desenvolver essas soluções, e um domínio de aplicação é a área ou tipo de problema em que o software será utilizado. Por exemplo, Python é uma linguagem que suporta o paradigma orientado a objetos e pode ser usada no domínio de desenvolvimento web.

3. Sintaxe ou semântica?

Classifique cada situação como problema principalmente de sintaxe ou de semântica:

Situação	Classificação
a) Um parêntese foi aberto e não foi fechado.	Sintaxe
b) O programa compila, mas deposita um valor negativo na conta errada.	Semântica
c) Uma palavra reservada foi escrita incorretamente.	Sintaxe
d) A condição do laço nunca se torna falsa.	Semântica
e) Uma função recebe dois argumentos, mas a chamada fornece apenas um.	Semântica
f) A fórmula da média usa a soma correta, mas divide pela quantidade errada.	Semântica

4. Etapas de tradução

Organize as etapas abaixo em uma sequência coerente para um processo de compilação: geração de código, análise léxica, execução, análise sintática, análise semântica. Depois explique, em uma frase, a função de cada etapa.

A sequência é: análise léxica → análise sintática → análise semântica → geração de código → execução. A análise léxica identifica os elementos do código, a sintática verifica sua estrutura, a semântica verifica seu significado, a geração de código produz o código executável e a execução realiza o programa.

Parte B - Paradigmas em ação

5. Identificação de paradigmas

Associe cada descrição ao paradigma predominante:

Descrição	Paradigma
a) O programa é uma sequência de comandos que altera o estado.	Imperativo
b) A solução é modelada com objetos que combinam dados e comportamentos.	Orientado a objetos
c) O resultado é obtido por composição e aplicação de funções.	Funcional
d) O conhecimento é descrito por fatos e regras, e o sistema realiza inferência.	Lógico
e) Várias tarefas progridem de forma sobreposta ou simultânea.	Concorrente

6. Curto-circuito

Considere a condição:

`x != 0 && 10 / x > 2`

a) O que acontece quando x vale zero em uma linguagem com avaliação em curto-circuito? b) O que poderia acontecer se a linguagem avaliasse obrigatoriamente as duas partes? c) Reescreva a lógica usando uma estrutura condicional explícita.

- a) Se $x = 0$, a segunda parte não é avaliada, evitando a divisão por zero.
- b) Poderia ocorrer um erro de divisão por zero.
- c) `if x != 0: if 10 / x > 2: ...`

7. Mesmo problema, estilos diferentes

Observe duas soluções em Python para somar os números pares de uma lista:

```
# Versão A
total = 0
for numero in numeros:
    if numero % 2 == 0:
        total = total + numero
```

```
# Versão B
total = sum(filter(lambda n: n % 2 == 0, numeros))
```

- a) Qual versão apresenta estilo mais imperativo? Por quê?
- b) Qual versão usa mais recursos funcionais?

c) Qual delas você considera mais legível para uma equipe iniciante? Justifique.

- a) A versão A é mais imperativa, pois utiliza comandos que alteram o estado de total durante um laço.
- b) A versão B utiliza mais recursos funcionais, como filter, lambda e sum.
- c) A versão A tende a ser mais legível para iniciantes por apresentar a lógica de forma mais explícita e sequencial.

Parte C - Escolhas e análise crítica

8. Escopo e tempo de vida

Analise o pseudocódigo:

```
x = 10
```

```
funcao calcular():
```

```
    y = 5
```

```
    se x > y:
```

```
        z = x + y
```

```
    mostrar(z)
```

```
    mostrar(y)
```

```
    mostrar(z)
```

Assumindo escopo léxico convencional, indique onde x, y e z podem ser acessados. A última instrução é válida? Explique.

x pode ser acessado dentro e fora da função, enquanto y e z podem ser acessados apenas dentro da função. A última instrução, mostrar(z), é inválida porque z foi declarado dentro do bloco condicional e não pode ser acessado fora dele.

9. Escolha de linguagem

Uma equipe precisa desenvolver um pequeno sistema de análise de dados. O prazo é curto, a equipe já conhece Python e o desempenho exigido é moderado. Outra pessoa defende usar uma linguagem de baixo nível porque 'é mais rápida'. Avalie a decisão considerando pelo menos cinco critérios.

A equipe deve considerar produtividade, conhecimento prévio, prazo, desempenho e facilidade de manutenção. Nesse caso, Python é uma escolha mais adequada, pois a equipe já o conhece, o prazo é curto e o desempenho exigido é moderado, tornando desnecessário optar por uma linguagem de baixo nível apenas pela maior velocidade potencial.

10. Trade-offs

Explique por que não existe uma linguagem objetivamente melhor em todos os aspectos. Apresente dois exemplos de trade-off, como produtividade versus controle, segurança versus desempenho, simplicidade versus expressividade ou concisão versus legibilidade.

Não existe uma linguagem melhor em todos os aspectos porque cada uma prioriza características diferentes. Por exemplo, uma linguagem pode oferecer maior produtividade, mas menor controle sobre o hardware, enquanto outra oferece maior controle e desempenho, mas exige mais complexidade e tempo de desenvolvimento.

Parte D - Aplicação e síntese

11. Linguagem multiparadigma

Escolha uma linguagem multiparadigma que você conhece. Mostre, com exemplos pequenos ou descrições, como ela permite utilizar pelo menos dois paradigmas diferentes.

Python é multiparadigma, pois permite programação imperativa, utilizando comandos e laços para alterar estados, e funcional, utilizando funções como map, filter e lambda para transformar e processar dados.

12. Inteligência artificial e responsabilidade

Uma ferramenta de IA gerou um programa que compila e produz uma saída aparentemente correta. Liste pelo menos seis verificações que um desenvolvedor responsável ainda deve realizar antes de usar esse código em produção.

O desenvolvedor deve verificar correção da lógica, segurança, tratamento de erros, desempenho, qualidade e legibilidade do código, uso adequado de dependências e possíveis casos extremos antes de utilizá-lo em produção.

13. Desafio prático

Implemente, em qualquer linguagem, um programa que receba uma lista de números e produza: (1) a soma dos pares; (2) a quantidade de valores positivos; e (3) o maior valor. Depois identifique o paradigma predominante da sua solução e explique como ela poderia ser reescrita em outro estilo.

```
numeros = [2, -3, 4, 7, -1, 8]
soma_pares = 0
quantidade_positivos = 0
maior = numeros[0]
for numero in numeros:
    if numero % 2 == 0:
        soma_pares += numero
    if numero > 0:
        quantidade_positivos += 1
    if numero > maior:
        maior = numero
print(soma_pares)
print(quantidade_positivos)
print(maior)
```

A solução utiliza predominantemente o paradigma imperativo, pois executa comandos sequencialmente e altera variáveis durante o processamento. Em um estilo funcional, poderia utilizar funções como filter e sum para realizar essas operações.

Síntese final

Escreva uma frase que resuma a ideia central da Aula 000. A resposta deve relacionar formas de pensar, escolha de ferramentas e qualidade da solução.

Conhecer diferentes paradigmas amplia as formas de pensar, ajuda a escolher as ferramentas mais adequadas e contribui para desenvolver soluções mais claras e de maior qualidade.